

Exercises: Introductory Course on Implicitly Switched ODEs and Filippov Sliding

July 18, 2025

Solution 1

- a) ode45 produces incorrect results, because its accuracy depends on the smoothness of the ODE's right-hand side. For equations with non-smooth or even discontinuous terms, the solver fails to capture the true solution behaviour as can be seen in the plot below. Your code should look as follows:

Right-hand side

File name: task1RHS.m

```
1 function dx = task1RHS(t, x, p)
2     dx = zeros(2, 1);
3     q1 = 50/27*t.^2 - 100/3*t + 400/3;
4
5     if x(1) <= 750
6         dx(1) = q1;
7     else
8         q2 = (1000-x(2)) * (p(1)*(t-19) + p(2));
9         dx(1) = q1 - q2;
10        if x(1) <= 800
11            dx(2) = -300;
12        end
13    end
14 end
```

Main (using ode45)

```
1 %% Task 1 Solution a)
2 initPaths();
3 integrator = @ode45;
4
5 tEnd = 30;
6 tSpan = [0 tEnd];
7 p = [3/4 1/4];
8 initvals = [0; 1000];
9
10 % (wrong) solution with ode45
11 options = odeset('RelTol', 1e-6, 'AbsTol', 1e-8);
12 sol_ode45 = ode45(@(t,x) task1RHS(t,x,p), tSpan, initvals, options);
13
14 T = 0:0.1:tEnd;
15 X_ode45 = deval(sol_ode45, T);
16 plot(T, X_ode45(1,:), 'b', T, X_ode45(2,:), 'r');
```

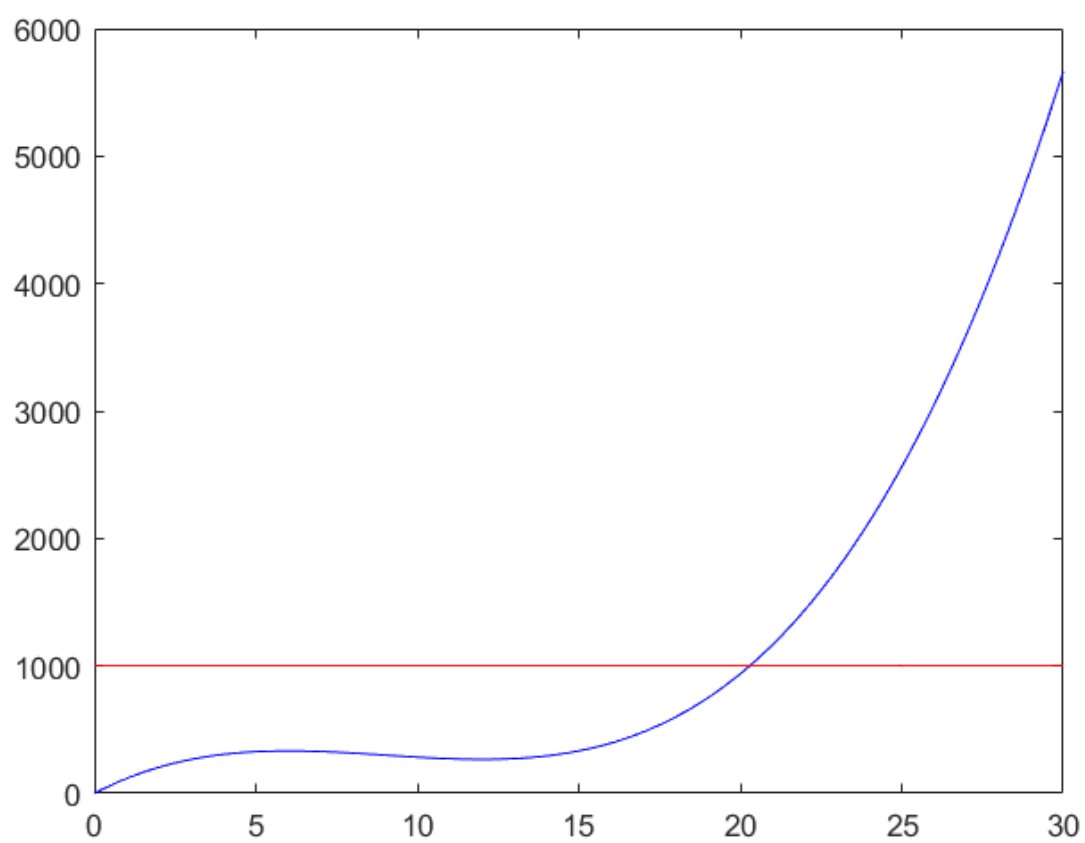


Figure 1: Plot with ode45

- b) The sensitivities are computed by adding this line in your main file. The sensitivities are wrong since they rely on the false solution from a)

```
1 %% Task 1 Sensitivities b)
2 plotSensitivities_ode45(sol, @(t,x,p) task1RHS(t,x,p), p, integrator, options)
;
```

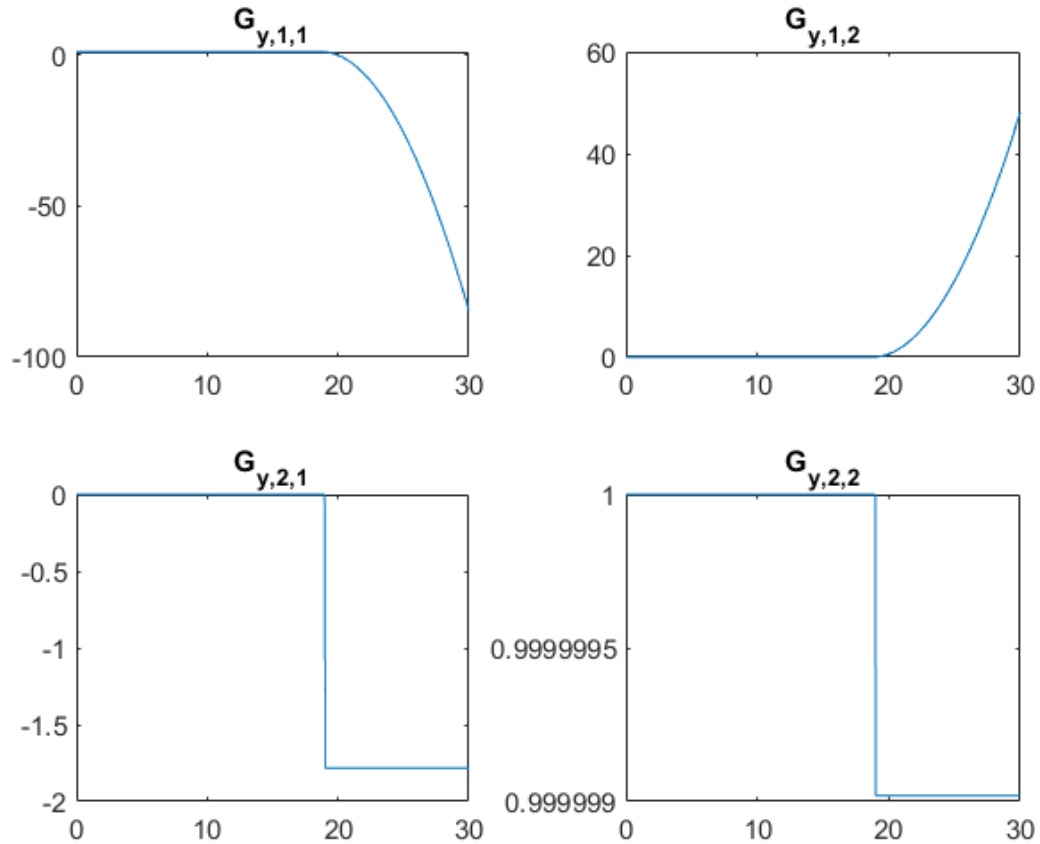


Figure 2: Sensitivities generated from the ode45 solution

- c) For the solution with IFDIFF, the RHS stays the same as in a). We need to create the solution structure by calling `solveODE()` which we pass the datahandle, time interval, initial values and parameters. The evaluation works the same as before. The main file should look like this:

```

1 %% Task 1 Solution c)
2 initPaths();
3 integrator = @ode45;
4
5 tEnd = 30;
6 tSpan = [0 tEnd];
7 p = [3/4 1/4];
8 initvals = [0; 1000];
9
10
11 % solution with ifdiff
12 datahandle = prepareDatahandleForIntegration('task1RHS', ...
13     'integrator', integrator, ...
14     'options', options);
15 sol_ifdiff = solveODE(datahandle, tSpan, initvals, p);
16
17 T = 0:0.1:tEnd;
18 X_ifdiff = deval(sol_ifdiff, T);
19 plot(T, X_ifdiff(1,:), 'b', T, X_ifdiff(2,:), 'r');

```

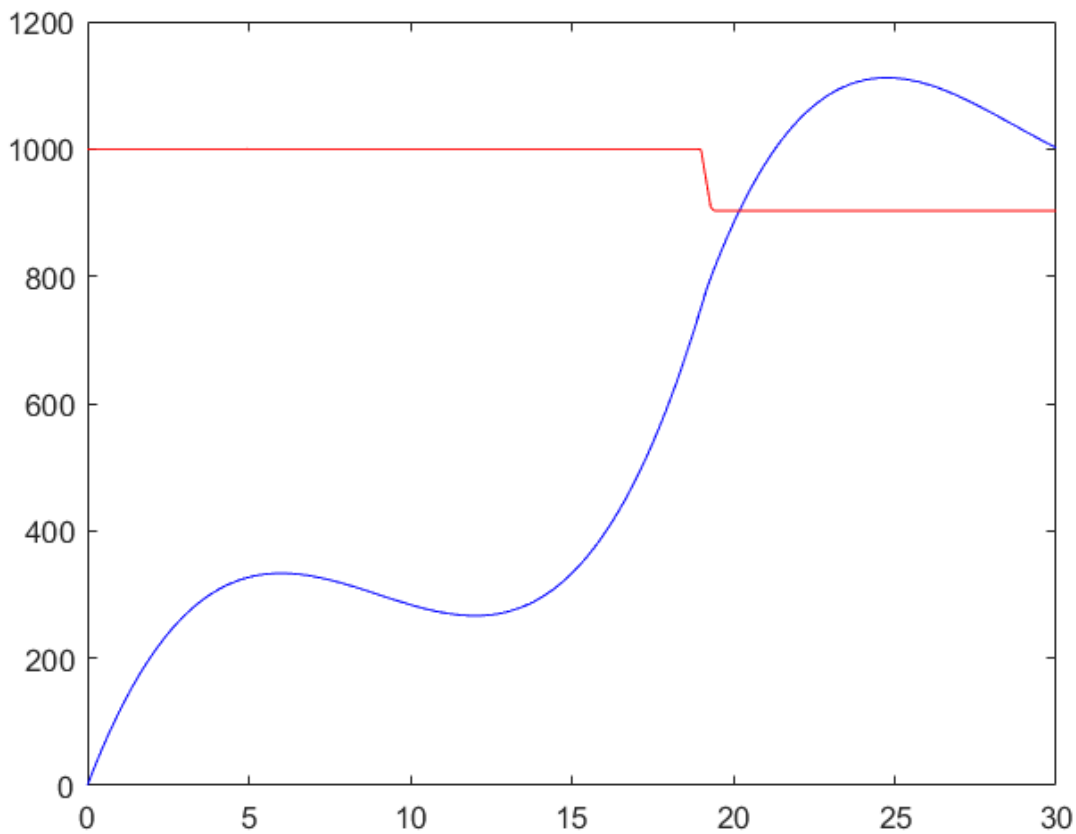


Figure 3: Solution for b) with IFDIFF

d) For the sensitivities with IFDIFF, add this line in your main file:

```
1 %% Task 1 Solution d)
2 plotSensitivities_ifdiff(datahandle, sol_ifdiff, p);
```

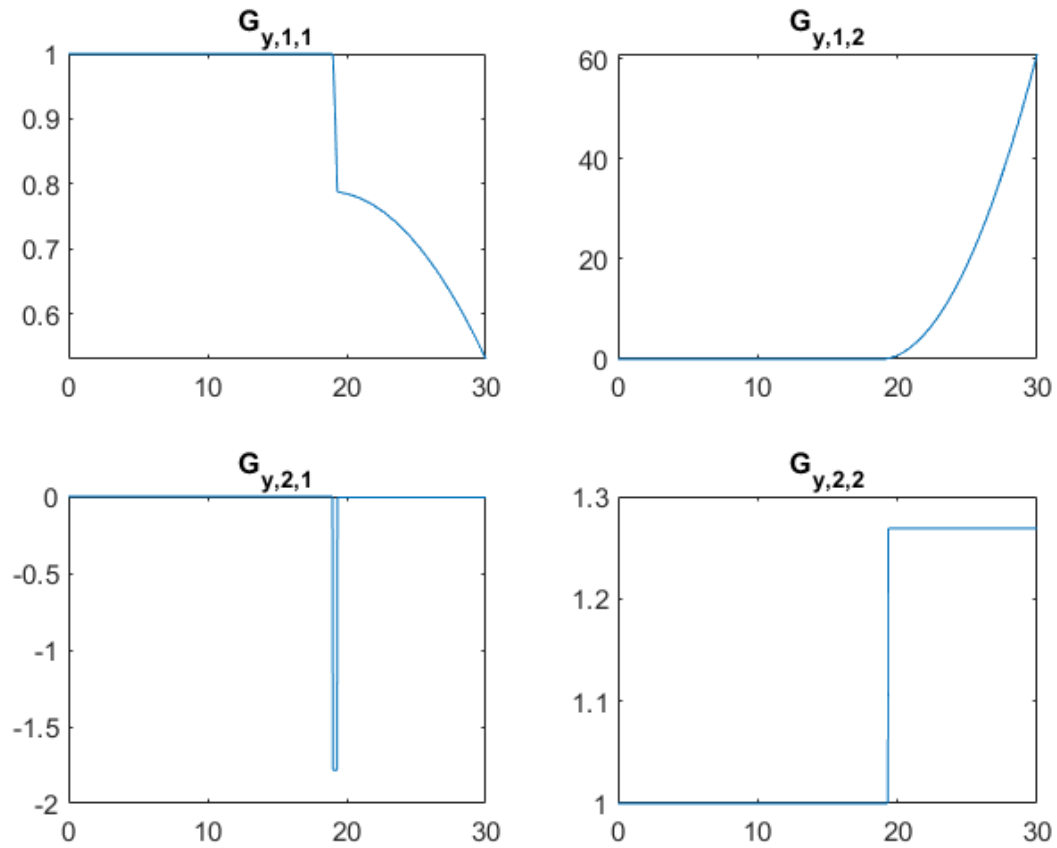


Figure 4: Sensitivities generated from the solution with IFDIFF

- e) The `MaxStep` solution looks, at first glance, better than one using `ode45`. However, after reaching the switching point, it cannot keep the accuracy of order 5, because it does not use the Jump-Update-formula as IFDIFF does. The implementation looks like this:

```

1 %% Task 1 Solution e)
2 initPaths();
3 integrator = @ode45;
4
5 tEnd = 30;
6 tSpan = [0 tEnd];
7 p = [3/4 1/4];
8 initvals = [0; 1000];
9
10 % solution with MaxStep
11 % setting the maximum allowed step size to 0.1
12 options = odeset('RelTol', 1e-6, 'AbsTol', 1e-8, 'MaxStep', 0.1);
13 sol_maxstep = ode45(@(t,x) task1RHS(t,x,p), tSpan, initvals, options);
14
15 T = 0:0.1:tEnd;
16 X_maxstep = deval(sol_maxstep, T);
17 plot(T, X_maxstep(1,:), 'b', T, X_maxstep(2,:), 'r');

```

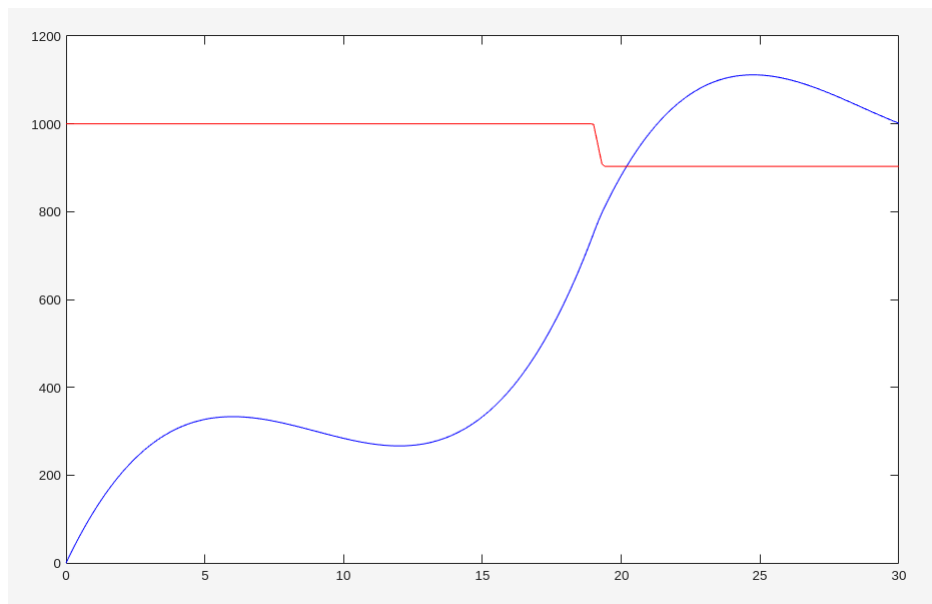


Figure 5: MaxStep Solution

- f) By adding this line to the main script for the MaxStep solution, we can observe the sensitivities. They are wrong, because the MaxStep solution does not provide the exact switching point detection as IFDIFF does.

```

1 %% Task 1 Sensitivities f)
2 plotSensitivities_ode45(sol_maxstep, @(t,x,p) task1RHS(t,x,p), p, integrator,
    options);

```

line to the code of e), we can take a look at the sensitivities, which are not correct.

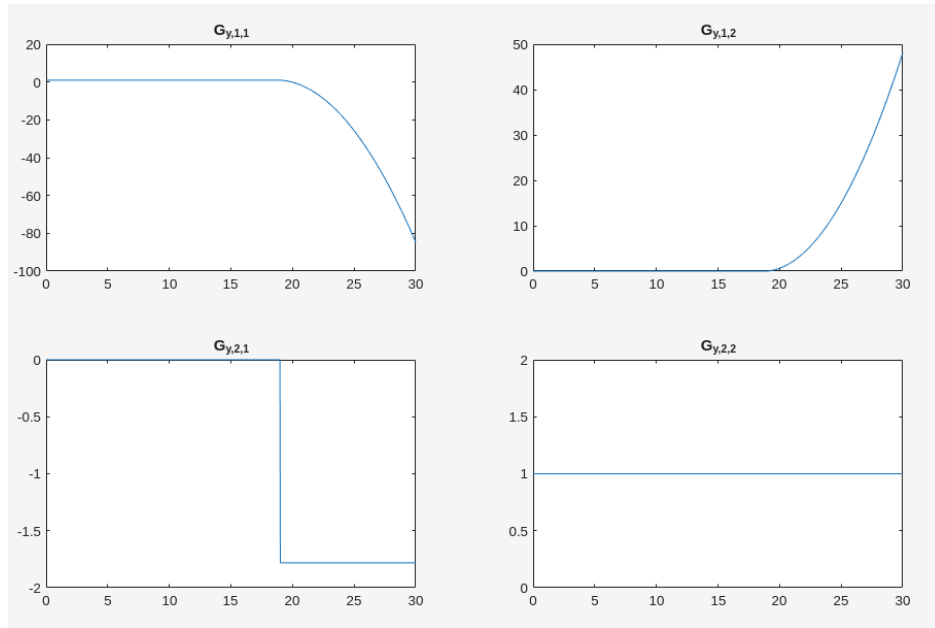


Figure 6: (wrong) sensitivities generated from the MaxStep solution

Solution 2

a) Right-hand side:

```
1 function dx = task2RHS(t, x, p)
2   dx = zeros(2, 1);
3   dx(1) = -sin(t);
4   dx(2) = abs(x(1));
5
6   % Jump when x_1 crosses p from below.
7   if ifdiff_jumpif(x(1) - p, 1)
8       % Flip sign of x_2: [x_2 + (-2*x_2) = -x_2].
9       ifdiff_update([0; -2*x(2)]);
10  end
11  end
```

Main script (without sensitivities):

```
1 %% Setup
2 % RHS
3 tspan = [0, 40];
4 y0 = [1; 0];
5 p = 0.5;
6
7 % Solver
8 integrator = @ode45;
9 options = odeset('AbsTol', 1e-10, 'RelTol', 1e-8);
10
11 %% Solve with IFDIFF
12 dh = prepareDatahandleForIntegration( ...
13     @task2RHS, ...
14     'integrator', integrator, ...
15     'options', options);
16 sol = solveODE(dh, tspan, y0, p);
17
18 %% Plot solution
19 fig = figure(1111);
20 clf(fig);
21 ax = axes(fig);
22
23 tdist = 0.01;
24 tplot = tspan(1):tdist:tspan(end);
25 yplot = deval(sol, tplot);
26 plot(ax, tplot, yplot);
```

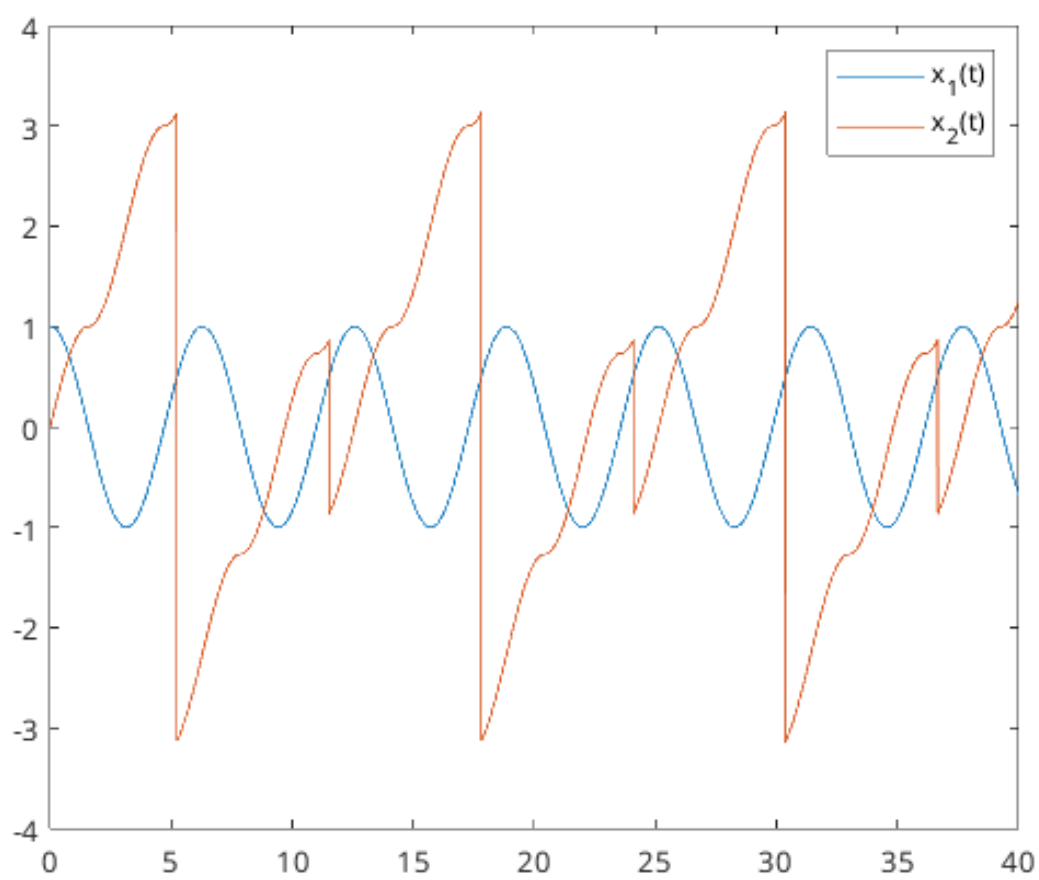



Figure 7: Task 2a: Solution with IFDIFF

b) Main script (with sensitivities):

```
1 %% Setup
2 % RHS
3 tspan = [0, 40];
4 y0 = [1; 0];
5 p = 0.5;
6
7 % Solver
8 integrator = @ode45;
9 options = odeset('AbsTol', 1e-10, 'RelTol', 1e-8);
10
11 %% Solve with IFDIFF
12 dh = prepareDatahandleForIntegration( ...
13     @task2RRHS, ...
14     'integrator', integrator, ...
15     'options', options);
16 sol = solveODE(dh, tspan, y0, p);
17
18 %% Plot solution
19 fig = figure(1111);
20 clf(fig);
21 ax = axes(fig);
22
23 tdist = 0.01;
24 tplot = tspan(1):tdist:tspan(end);
25 yplot = deval(sol, tplot);
26 plot(ax, tplot, yplot);
27
28 %% Compute Sensitivities
29 dim_y = size(sol.y, 1);
30 dim_p = length(p);
31 FDstep = generateFDstep(dim_y, dim_p);
32
33 sensFun = generateSensitivityFunction(dh, sol, FDstep, 'method', 'VDE');
34
35 %% Plot sensitivities
36 plotSensitivitiesSwitched(sol, sensFun, tspan);
```

Explanations for sensitivity plots (below):

Let us first examine the sensitivities of the first component x_1 :

- $G_{y,1,1}$: Since x_1 is always a cosine, the sensitivity of x_1 w.r.t. the initial value of x_1 ($G_{y,1,1}$) is equal to 1 everywhere (apart from some numerical inaccuracy), i.e. slightly disturbing the initial value changes the solution trajectory in the same manner.
- $G_{y,1,2}$: The sensitivity of x_1 w.r.t. the initial value of x_2 ($G_{y,1,2}$) is equal to zero everywhere, because x_1 is completely independent from the second component x_2 .
- $G_{p,1,1}$: Similarly, the sensitivity of x_1 w.r.t. the parameter p ($G_{p,1,1}$) is equal to zero everywhere (apart from some numerical inaccuracy), since x_1 does not depend on the parameter.

Now let us consider the sensitivities of the second component x_2 :

- $G_{y,2,1}$: The sensitivity of x_2 w.r.t. the initial value of x_1 ($G_{y,2,1}$) exhibits a "spiky" behavior with jumps and is somewhat difficult to analyze. Slightly increasing the initial value of x_1 moves the initial value of the derivative of x_1 , which as a reminder is $\dot{x}_1(t) = -\sin(t)$. This affects the growth of the solution segment before the first jump to varying degrees depending on the magnitude of the sine at the time of interest, i.e. at $t = 0$ where the sine is 0, the effect is minimal, whereas the effect at the time points where the sine has its peaks and valleys is the largest.

The effect on the solution segment after the first jump is additionally amplified, because increasing the initial value, makes the jump occur earlier. This in turn makes the solution segment before the jump reach a lower peak, so the solution segment after the first jump starts at a higher value, which is why the "spikes" are shifted upwards for the solution segment after the first jump. However, the second jump is also shifted, so the bigger starting value of the solution segment after the first jump is balanced out by an earlier jump giving the second solution segment less time to grow. Ultimately, this causes the sensitivity after the second jump to match the sensitivities before the first jump. Due to the periodicity of the solution, this pattern repeats.

$G_{y,2,2}$: The sensitivity of x_2 w.r.t. the initial value of x_2 ($G_{y,2,2}$) alternates between 1 and -1 after each jump. The growth of x_2 only depends on x_1 , so slightly disturbing the initial value before a jump has occurred results in an equivalent change in the solution trajectory. However, after the first jump, the effect of a disturbance in the initial value is inverted:

If we let our solution climb to a higher peak before the first jump by increasing the initial value, then the solution segment after the first jump will start at a smaller value. Therefore, the disturbance of the initial value has an inverse effect on the solution segment after the first jump (-1) compared to the solution segment before the first jump (+1). However, because the solution after the first jump now starts at a lower value, it will also climb to a lower peak before the second jump, making the solution segment after the second segment start a bigger value, which once again inverts the effect of a disturbance in the initial value (+1). Due to the periodicity of the solution, this pattern repeats.

$G_{p,2,1}$: The sensitivity of x_2 w.r.t. the parameter p ($G_{p,2,1}$) alternates between 0 and around -1.15 after each jump. Slightly increasing the parameter shifts the first jump to the right. This has no immediate effect on the solution segment before the first jump. However, it does allow the solution segment before the first jump to reach a higher peak, which means that the solution segment after the first jump starts at a smaller value, shifting the second solution segment downwards. But, since the second jump is also shifted to the right, the solution segment after the first jump essentially manages to reach the same peak as before, which means that the solution segment after the second jump starts at the same value. Therefore, the solution segment after the second jump is not affected by a slight parameter disturbance, i.e. the sensitivity is (close) to zero. Due to the periodicity of the solution, this pattern repeats.

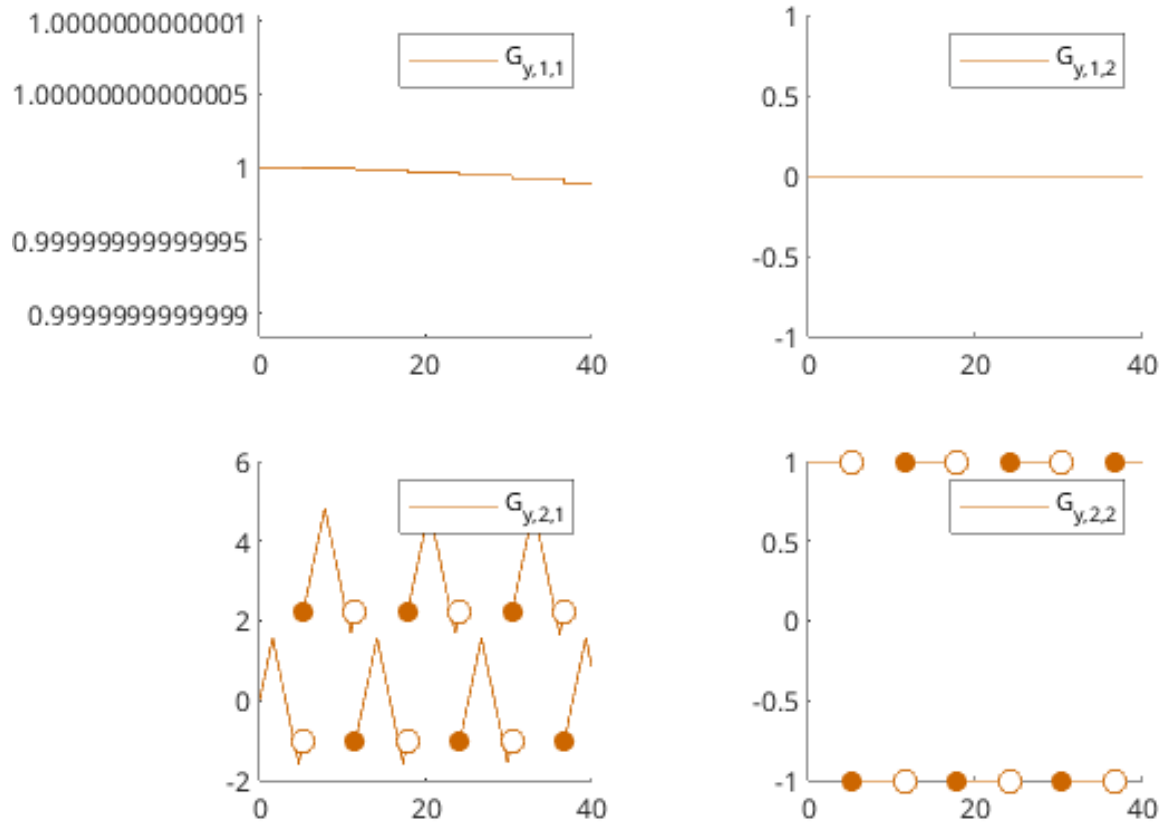


Figure 8: Task 2b: Sensitivities w.r.t. initial values

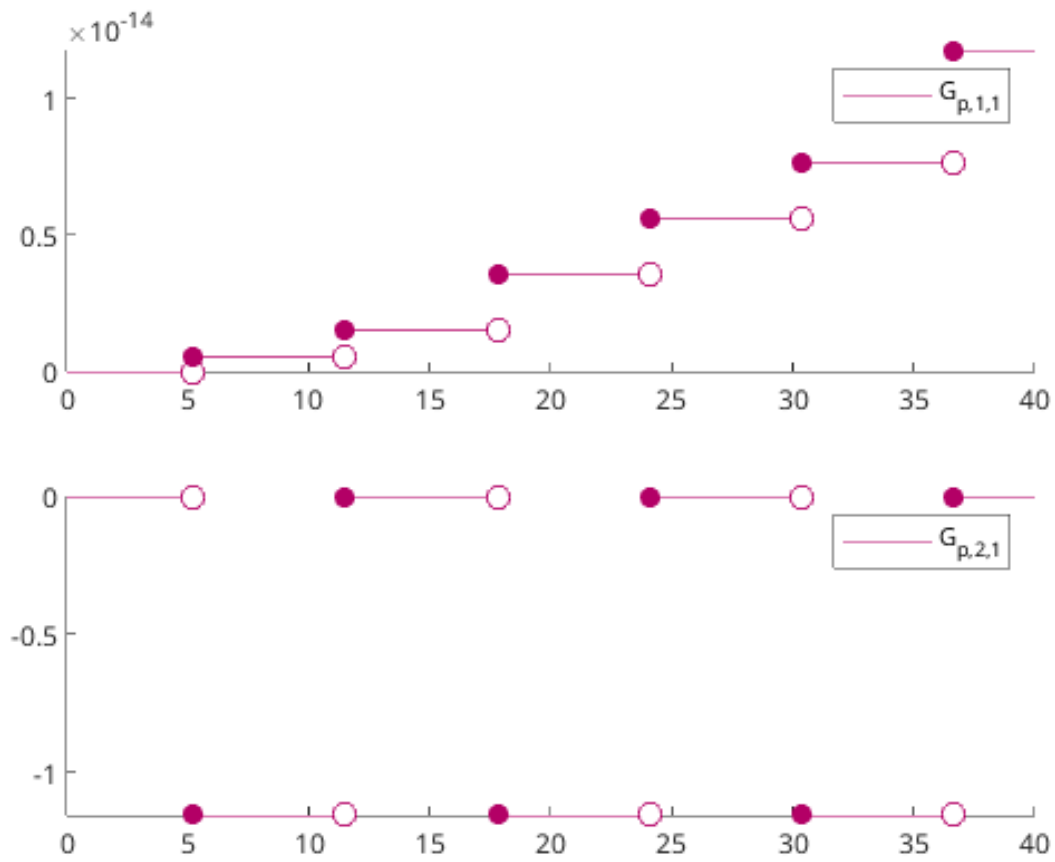


Figure 9: Task 2b: Sensitivities w.r.t. parameters

Solution 3: Filippov Sliding

We store the RHS inside a function file named `task3RHS.m` which looks like the following. Notice that we code the switching behavior via the switching manifold.

```
1 function dx = task3RHS(t,x,p)
2 % preallocate output
3 dx = [0 ; 0 ; 0];
4
5 % x = (p1, p2, P)    % prey, prey2, predator
6 p1 = x(1);
7 p2 = x(2);
8 P  = x(3);
9
10 % p = (r1, r2, beta1, beta2, q1, q2, m, e, aq) % parameters
11 r1  = p(1);
12 r2  = p(2);
13 beta1 = p(3);
14 beta2 = p(4);
15 q1  = p(5);
16 q2  = p(6);
17 m   = p(7);
18 e   = p(8);
19 aq  = p(9);
20
21 % switching manifold
22 H = beta1*p1 - aq*beta2*p2;
23 % switched RHS
24 if H >= 0
25 dx(1) = (r1 - beta1*P) .* p1;
26 dx(2) = r2*p2;
27 dx(3) = (e*q1*beta1*p1 - m) .* P;
28 else
29 dx(1) = r1*p1;
30 dx(2) = (r2 - beta2*P) .* p2;
31 dx(3) = (e*q2*beta2*p2 - m) .* P;
32 end
33
34 end
```

The main file contains the setup of the parameters, initial values, the plotter and the solution with Matlab as well as the solution with IFDIFF and the according plot specifications.

```
1 %% SETUP
2 % parameter values
3 % fraction of prey in refuge:
4 m = 0.845;
5 % growth rates of:
6 r1 = 1.124; % predator
7 r2 = 0.357; % prey
8 % conversion efficiency:
9 e = 0.676;
10 % catchability coefficients:
11 q1 = 0.694;
12 q2 = 1.387;
13 % functional response parameters:
14 aq = 0.634;
15 beta1 = 7.9;
16 beta2 = 1.925
17
```

```

18
19 % initial values, parameter vector
20 a = 0.28; % initial value
21 x0_1 = [a ; a ; r1-r2]; % starting point of the solution trajectory
22 p = [r1, r2, beta1, beta2, q1, q2, m, e, aq];
23
24 % time interval,
25 tspan = [0 200];
26
27 % solver selection and configuration
28 intMatlab = @ode45;
29 intIfdiff = intMatlab;
30 intOptions = odeset('reltol', 1e-5, 'abstol', 1e-5, 'MaxStep', 0.5);
31
32
33 % configure the plotter function
34 X_plot = linspace(tspan(1), tspan(end), 1000);
35 fignum = 1000;
36 figure(fignum); clf; hold('on');
37 % plotter function
38 function h = plotter(fignum, x, y, color, name, lw)
39     figure(fignum); hold on;
40     h = plot3(y(3,:), y(2,:), y(1,:), 'Color', color, 'LineWidth', lw, 'DisplayName',
41         name);
42     view([97 51]);
43     grid on;
44     box on;
45     xlabel('Predator');
46     ylabel('Prey 2');
47     zlabel('Prey 1');
48     legend('location', 'northeast');
49     drawnow
50     pause(1.0);
51     set(fignum, 'Position', [200 250 750 375]);
52 end
53
54 plotit = @plotter;
55
56 %% Optional: Name generators
57 nameIfdiff = @(f) sprintf('ifdiff/%s', func2str(f));
58 namePlain = @(f) sprintf('plain %s', func2str(f));
59
60 %% Optional: Mark the initial point
61 if true
62     xx = x0_1;
63     hStart = plot3(xx(3), xx(2), xx(1), 'k.', 'MarkerSize', 25, 'DisplayName', 'x_0');
64 end
65
66 %% Plain integration, no treatment of switches
67 fprintf('Integrating with plain %s ...\n', func2str(intMatlab))
68 figure(fignum);
69 sol_matlab = intMatlab(@(t,x) pprhs(t,x,p), tspan, x0_1, intOptions);
70 X_matlab = X_plot;
71 Y_matlab = deval(sol_matlab, X_matlab);
72 linewidth = 1.0;
73 hMatlab = plotit(fignum, X_matlab, Y_matlab, 'r', namePlain(intMatlab), linewidth)
74 ;
75

```

```

74 %% IFDIFF
75 fprintf('Integrating with IFDIFF/%s ...\n', func2str(intIfdiff))
76 figure(fignum);
77 datahandle = prepareDatahandleForIntegration('task3RHS', 'solver', func2str(
78     intIfdiff), 'options', intOptions);
79 sol_ifdiff = solveODE(datahandle, tspan, x0_1, p);
80 X_ifdiff = X_plot;
81 Y_ifdiff = deval(sol_ifdiff, X_ifdiff);
82 linewidth = 3.0;
83 hIFDIFF = plotit(fignum, X_ifdiff, Y_ifdiff, 'g', nameIfdiff(intIfdiff), linewidth
    );

```

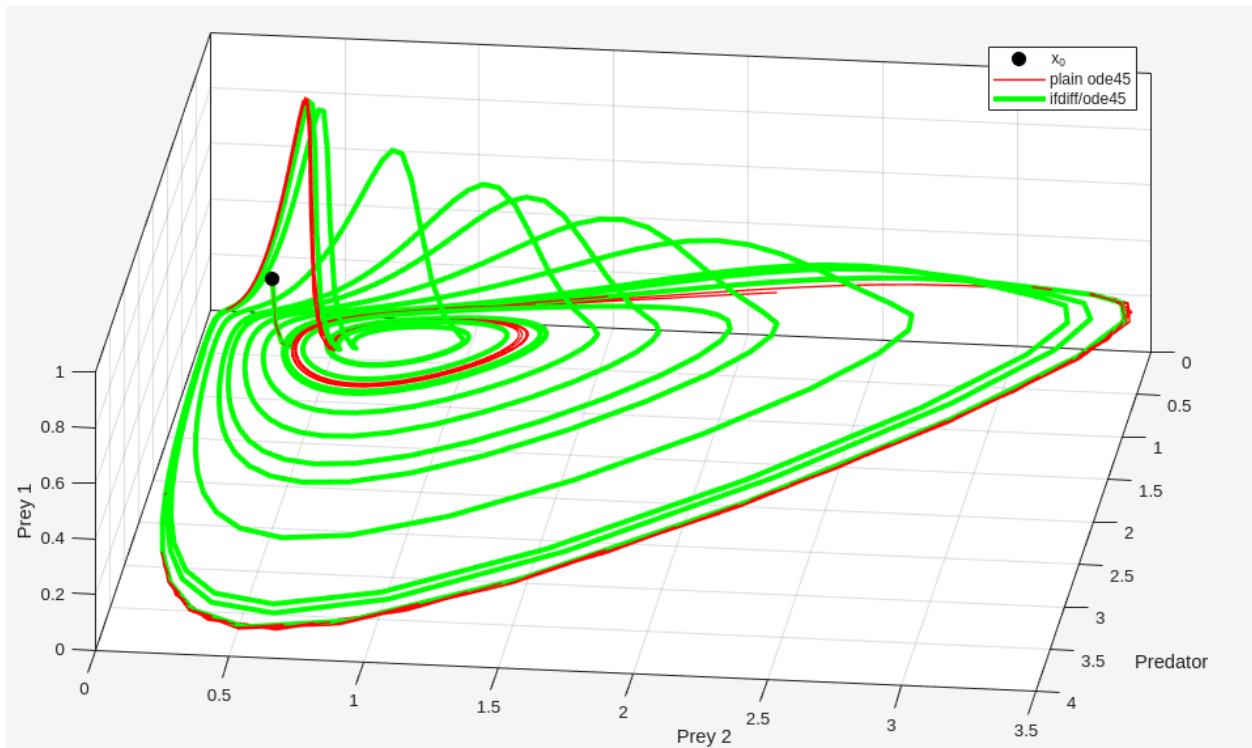


Figure 10: Solution plot to task 3. As you can see, the solution trajectory ode45 computes (red) differs from the IFDIFF solution (green). By solving the system via Explicit Euler, we can verify that IFDIFF is correct!

Solution 4: Parameter Estimation

We store the RHS inside a function file named `LotkaVolterraRHS.m` which looks like this: (Note: As always the RHS file needs the signature `func(t, x, p)`. However, Lotka Volterra is inherently autonomous and doesn't need `t`. To avoid warnings by MATLAB, instead of `t` we place `~`)

```
1 function dx = LotkaVolterraRHS(~,x,p)
2     alpha = p(1); % 0.2
3     beta = p(2); % 0.01
4     gamma = p(3); % 0.001
5     delta = p(4); % 0.1
6     dx = zeros(2,1);
7     if x(1) > 60
8         dx(1) = alpha * x(1) - beta * x(1) * x(2);
9         dx(2) = 5 * gamma * x(1) * x(2) - 0.2 * delta * x(2);
10    else
11        dx(1) = alpha * x(1) - beta * x(1) * x(2);
12        dx(2) = gamma * x(1) * x(2) - delta * x(2);
13    end
14 end
```

The main part of the exercise is solved here (although we added another plot for the first iterate of `lsqnonlin`)

```
1 %% Solution ifdiff
2 initPaths();
3 integrator = @ode45;
4 odeoptionsrhs_test = odeset('AbsTol', 1e-14, 'RelTol', 1e-6);
5 datahandle = prepareDatahandleForIntegration('LotkaVolterraRHS', 'integrator',
6     func2str(integrator), 'options', odeoptionsrhs_test);
7
8 tspan = [0 120];
9 initialvalues = [20; 10];
10 parameters_ODE = getParams_LV();
11 sol = solveODE(datahandle, tspan, initialvalues, parameters_ODE);
12
13 figure(1)
14 plot(sol.x, sol.y(1,:), 'LineWidth', 3, 'color', [0.08,0.05,0.68]);
15 hold on;
16 plot(sol.x, sol.y(2,:), 'LineWidth', 3, 'color', [0.08,0.05,0.68]);
17
18 %% Precalculations for finite differences for sensitivities
19 dim_y = size(sol.y, 1);
20 dim_p = length(parameters_ODE);
21 FDstep = generateFDstep(dim_y, dim_p);
22
23 %% Generation of synthetic measurements
24 t = tspan(1):tspan(end);
25 sigma = 3;
26 rng(0);
27 rand = randn(dim_y * length(t), 1);
28 measurements = reshape(deval(sol, t), [], 1) + sigma.*rand;
29
30 %% Generation of the residual function
31 integrator_residual = @solveODE;
32 RHS = @LotkaVolterraRHS;
33 method = 'VDE';
34 residual_function = generateResFunction(t, datahandle, sol, measurements, tspan,
35     parameters_ODE, FDstep, method);
```

```

36 %% Parameter estimation
37 options = optimoptions('lsqnonlin','SpecifyObjectiveGradient',true, 'Algorithm','
    trust-region-reflective', 'Display', 'iter', 'DerivativeCheck', 'off', '
    typicalX', [parameters_ODE;1;1]);
38 parameters_init = [1.2*parameters_ODE; measurements(1:dim_y)];
39 tic;
40 [param_opt,resnorm,residual,exitflag,output,lambda,jacobian] = lsqnonlin(
    residual_function, parameters_init, zeros(6,1), [1, 1, 1, 1, 30, 30], options);
41 toc;
42 percent = (param_opt*100./[getParams_LV();20;10])-100;
43 disp(param_opt);
44
45 %% Solution with estimated parameters
46 initialvalues_opt = param_opt(5:6);
47 parameters_ODE_opt = param_opt(1:4);
48 sol_opt = solveODE(datahandle, tspan, initialvalues_opt, parameters_ODE_opt);
49
50 %% Trajectory before estimation
51 init_val = measurements(1:dim_y);
52 sol_prev = solveODE(datahandle, tspan, init_val, 1.2*parameters_ODE);
53
54 %% Plot of real solution and of solution with estimated parameters
55 measurements_plot = reshape(measurements, 2, []);
56 figure(2)
57 T = 0:0.1:120;
58 y = deval(sol,T);
59 y_opt = deval(sol_opt, T);
60 y_prev = deval(sol_prev, T);
61 %
62 plot(sol_prev.x, sol_prev.y(1,:), ':', 'LineWidth', 1, 'color', [0.08,0.05,0.68]);
63 hold on;
64 plot(sol_prev.x, sol_prev.y(2,:), ':', 'LineWidth', 1, 'color', [0.08,0.05,1.00]);
65 %
66 plot(T, y(1,:), 'LineWidth', 3, 'color', [0.08,0.05,0.68]);
67 hold on
68 plot(T, y_opt(1,:), '--', 'LineWidth', 3, 'color', [0.71,0.69,0.88]);
69 plot(t, measurements_plot(1,:), '*', 'MarkerSize', 5, 'color', [0.08,0.05,0.68]);
70 plot(T, y(2,:), 'LineWidth', 3, 'color', [0.01,0.30,0.01]);
71 plot(T, y_opt(2,:), '--', 'LineWidth', 3, 'color', [0.72,0.92,0.72]);
72 plot(t, measurements_plot(2,:), '*', 'MarkerSize', 5, 'color', [0.01,0.30,0.01]);
73 xlabel('time [t]')
74 ylabel('solution [y]')
75 set(gca, 'FontSize', 12);
76 set(gca, 'Box', 'off');
77 hold off
78 legend({'Prey init', 'Pred. init', '$y_L(t;\tilde{p}^{\ast})$', '$y_L(t;\tilde{p}^{\ast})$', 'meas. $Prey$', '$y_S(t;\tilde{p}^{\ast})$', '$y_S(t;\tilde{p}^{\ast})$', 'meas. $Predator$', 'Interpreter', 'LaTeX'});

```

Right at the heart of the optimization procedure we use a forward integration and its sensitivities to generate a residual vector as well as the corresponding jacobian matrix. This derivative information can then be used by lsqnonlin.

```

1 function [residual_function] = generateResFunction(t, datahandle, sol,
2     measurements, tspan, parameters_ODE, FDstep, method)
3     %Generation of residual function for parameter estimation using nested
4     functions
5
6     residual_function = @computeResidual;

```

```

5   dim_y = size(sol.y, 1);
6   dim_p = length(parameters_ODE);
7
8   return
9
10  function [residual, jacobian] = computeResidual(p)
11      %Residual function
12
13      initialvalues = p(dim_p+1:end);
14      params = p(1:dim_p);
15      sol_p = solveODE(datahandle, tspan, initialvalues, params);
16      h = reshape(deval(sol_p, t), [], 1);
17
18      residual = (h-measurements);
19
20      % Sensitivities
21      if nargout > 1
22          sensitivities_function = generateSensitivityFunction(datahandle, sol_p,
23              FDstep, 'method', method, 'p_ttyp', parameters_ODE);
24          sensitivites = sensitivities_function(t);
25          jacobian = zeros(dim_y*length(t), dim_p + dim_y);
26          count = 1;
27          for i = 1:dim_y:dim_y*length(t)
28              jacobian(i:i+(dim_y-1), 1:dim_p) = sensitivites(count).Gp;
29              jacobian(i:i+(dim_y-1), (dim_p+1):end) = sensitivites(count).Gy;
30              count = count + 1;
31          end
32      end
33  end

```

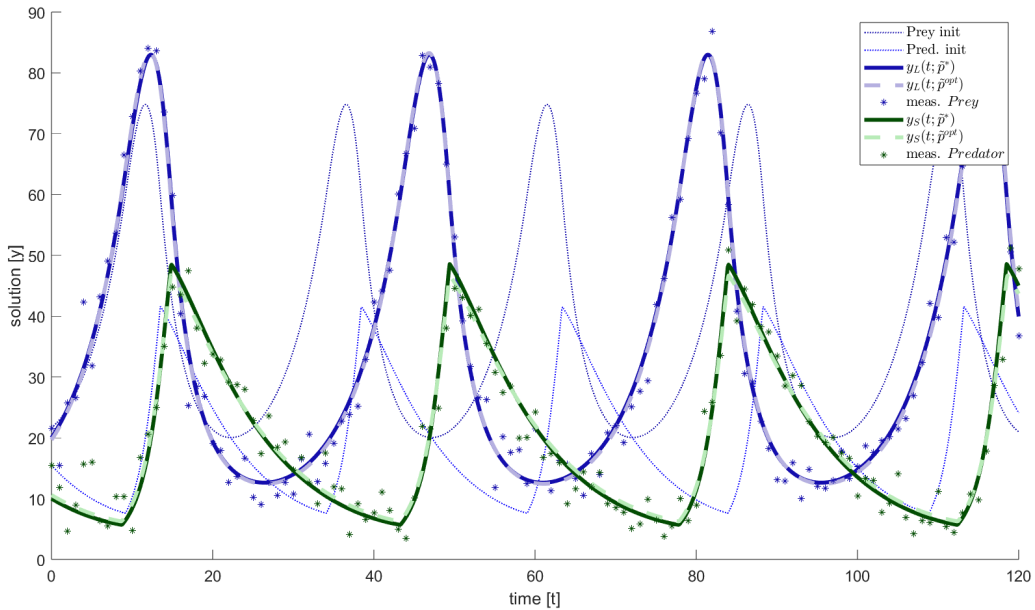


Figure 11: Trajectories from "real" solution, initial guess of PE, synthetic measurements and estimated solution